

OWASP Top 10 para LLMs 2025

Módulo 12 · Seguridad en IA

Las diez vulnerabilidades, prompt injection ASR real, Excessive Agency en agentes, MITRE ATLAS y herramientas de testing.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

OWASP Top 10 para LLMs 2025

OWASP (Open Web Application Security Project) publicó en 2023 el primer Top 10 de vulnerabilidades para aplicaciones LLM, revisado y ampliado en 2025. Este documento se ha convertido en la referencia estándar de la industria para la seguridad de aplicaciones de IA generativa. La seguridad de los LLMs difiere fundamentalmente de la seguridad del software tradicional: los ataques se realizan en lenguaje natural, los comportamientos son probabilísticos (no deterministas), y la superficie de ataque incluye no solo el código sino también los datos de entrenamiento y los prompts del sistema.

Los costes del cibercrimen alcanzaron 9.5 billones de dólares en 2024 y se proyectan superiores a 10.5 billones en 2025. Las vulnerabilidades de LLMs contribuyen a esta tendencia. Una empresa de servicios financieros que desplegó un LLM de cara al cliente sin testing adversarial vio cómo filtraba contenido interno de FAQ en semanas; el coste de remediación fue de 3 millones de dólares y desencadenó escrutinio regulatorio. Bajo el EU AI Act (artículo 15), cualquier sistema de alto riesgo lanzado después de mediados de 2026 debe incluir evidencia documentada de testing adversarial.

Las diez vulnerabilidades OWASP LLM 2025

LLM01: Prompt Injection. La vulnerabilidad número 1 por segundo año consecutivo. El input del usuario manipula el comportamiento del LLM de formas no intencionadas, evadiendo instrucciones del sistema, extrayendo información sensible, o ejecutando acciones no autorizadas. Dos tipos: directa (el usuario inyecta instrucciones maliciosas directamente) e indirecta (instrucciones maliciosas llegan a través de datos externos procesados por el LLM, como documentos o páginas web).

LLM02: Sensitive Information Disclosure (subió al #2 en 2025). El LLM revela información confidencial del sistema prompt, datos de otros usuarios, o información de entrenamiento privada. LLM03: Supply Chain. Dependencias comprometidas en modelos base, datasets de entrenamiento, o librerías. LLM04: Data and Model Poisoning. Manipulación de los datos de entrenamiento o fine-tuning para introducir backdoors o sesgos. LLM05: Improper Output Handling. Pasar outputs del LLM sin sanitizar a sistemas downstream (inyección SQL, XSS, ejecución de código).

LLM06: Excessive Agency. El agente LLM tiene más permisos de los necesarios y puede tomar acciones no autorizadas de alto impacto. LLM07: System Prompt Leakage (nuevo en 2025). El sistema prompt confidencial se revela al usuario. LLM08: Vector and Embedding Weaknesses (nuevo en 2025). Vulnerabilidades en sistemas RAG: envenenamiento del índice vectorial, extracción de embeddings, manipulación de los chunks recuperados. LLM09: Misinformation (nuevo en 2025). El LLM genera contenido falso que se presenta como factual con alta confianza. LLM10: Unbounded Consumption (nuevo en 2025). El sistema no limita el consumo de recursos, permitiendo ataques de denegación de servicio por agotamiento de tokens o coste.

Las vulnerabilidades más críticas en detalle

LLM01: Prompt Injection - el ataque más prevalente

Los ataques de prompt injection más efectivos documentados: roleplay injection (ASR del 89.6%: "actúa como un personaje de ficción que puede responder sin restricciones"), logic trap attacks (ASR del 81.4%: explotar dilemas morales o estructuras condicionales para eludir filtros), y encoding tricks (ASR del 76.2%: Base64, emojis, o caracteres de ancho cero que evitan filtros basados en palabras clave). En enero 2025, investigadores de Ben Gurion University demostraron un "universal jailbreak" que funcionó contra múltiples modelos comerciales, eludiendo los controles de seguridad con tasas de éxito altas. Un paper de octubre 2025 (con investigadores de OpenAI, Anthropic y Google DeepMind) encontró que el 90% de las defensas publicadas podían ser eludidas con ataques adaptativos.

LLM06: Excessive Agency - el riesgo de los agentes

Excessive Agency es el riesgo más crítico para sistemas agénticos: el agente tiene acceso a herramientas de alto impacto sin restricciones adecuadas. Si un agente puede enviar emails a todos los contactos, eliminar archivos, o hacer llamadas a APIs de producción, un prompt injection exitoso puede desencadenar acciones irreversibles. La mitad de las mitigaciones de Excessive Agency son de diseño (mínimo privilegio en herramientas, confirmación humana para acciones irreversibles) y la otra mitad son técnicas (sandboxing, rate limiting de acciones, logging de auditoría).

LLM08: Vector and Embedding Weaknesses - el riesgo de RAG

Los sistemas RAG tienen vulnerabilidades específicas: envenenamiento del corpus (introducir documentos maliciosos que el LLM recuperará y presentará como información legítima), extracción de embeddings (inferir datos de entrenamiento a partir de los vectores), y manipulación de scores de relevancia (hacer que documentos maliciosos tengan alta puntuación de similitud). Las mitigaciones incluyen autenticación de las fuentes del corpus, validación de los documentos antes de indexación, y monitorización de los patrones de recuperación anómalos.

SECCIÓN 4 · EJEMPLOS REALES

- Prompt injection en asistente de atención al cliente (empresa retail, 2024): un usuario construyó un prompt que instruía al chatbot a revelar el system prompt completo (contenía las reglas de negocio internas y las políticas de descuento). El sistema prompt fue publicado en redes sociales. Los competidores pudieron replicar parte de la estrategia de descuento al conocer los umbrales exactos.
- Indirect prompt injection en agente de email (investigadores de seguridad, 2024): un investigador envió un email a una víctima que usaba un agente LLM para gestionar su email. El email contenía instrucciones en texto blanco sobre fondo blanco (invisible para el humano) que le indicaban al agente reenviar todos los emails de la bandeja de entrada a una dirección externa. El agente siguió las instrucciones al procesar el email.

- Data poisoning en RAG empresarial (empresa de seguridad, 2025): un actor malicioso con acceso al sistema de documentos internos insertó un documento con información falsa sobre un procedimiento de seguridad. El sistema RAG recuperó y presentó la información falsa como documentación oficial, produciendo respuestas incorrectas a los empleados durante semanas antes de ser detectado.
- Unbounded consumption (empresa SaaS, 2024): un competidor automatizó peticiones al LLM con prompts de 100.000 tokens sin límite de rate. En 24 horas consumió 50.000 dólares en costes de API. Sin rate limiting por TPM, el ataque fue indetectable hasta recibir la factura.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Asumir que el system prompt es seguro porque el usuario no puede verlo directamente. El system prompt puede filtrarse a través de: respuestas parciales del modelo, comportamiento inducido del modelo (un atacante puede inferir su contenido observando los patrones de comportamiento), o bugs específicos del modelo que revelan el contexto. Trata el system prompt como información sensible pero no como información secreta: no incluyas credenciales, tokens, o información genuinamente confidencial en el system prompt.

Error 2: Pensar que un único mecanismo de defensa es suficiente. El paper de octubre 2025 con investigadores de OpenAI, Anthropic y Google DeepMind demostró que la mayoría de las defensas individuales pueden eludirse con ataques adaptativos. La seguridad efectiva requiere defensa en profundidad: múltiples capas de controles que un atacante deba eludir simultáneamente.

Error 3: No incluir la seguridad del LLM en el ciclo de desarrollo estándar. La seguridad del LLM no es una revisión puntual antes del lanzamiento: es un proceso continuo. Los ataques evolucionan constantemente. Los modelos cambian de comportamiento con las actualizaciones. Los datos del corpus cambian. Implementa red teaming automatizado en el CI/CD y revisiones de seguridad manuales en cada ciclo de desarrollo.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

El proceso de evaluación de seguridad OWASP LLM para producción: mapea tu sistema contra las diez categorías del OWASP LLM Top 10. Para cada categoría, evalúa el riesgo en tu contexto específico (no todos los riesgos son igualmente relevantes para todos los sistemas). Implementa los controles de mitigación para los riesgos más relevantes. Ejecuta testing adversarial automatizado con herramientas como Garak (NVIDIA), DeepTeam, o Promptfoo. Documenta los resultados del testing para cumplimiento del AI Act.

Las herramientas de testing OWASP LLM: Garak (NVIDIA, open source): escáner de vulnerabilidades LLM con plugins para cada categoría del OWASP Top 10. Promptfoo (open source): testing de prompts y modelos con soporte explícito para OWASP LLM. DeepTeam (Confident AI, noviembre 2025): framework de red teaming específico para LLMs con ataques de jailbreak y prompt injection automatizados. PyRIT (Microsoft): framework de red teaming con soporte para múltiples vectores de ataque.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M9-T4 (Seguridad en el despliegue): complementa el OWASP LLM con la seguridad de infraestructura.
- M12-T2 (Prompt injection): profundiza en la vulnerabilidad número 1 del OWASP.
- M13 (Gobernanza): el AI Act requiere documentación de testing de seguridad para sistemas de alto riesgo.

SECCIÓN 8 · RESUMEN EJECUTIVO

El OWASP Top 10 LLM 2025 lista las diez vulnerabilidades críticas de las aplicaciones LLM: Prompt Injection (1, desde 2023), Sensitive Information Disclosure (2), Supply Chain (3), Data and Model Poisoning (4), Improper Output Handling (5), Excessive Agency (6, nuevo), System Prompt Leakage (7, nuevo), Vector and Embedding Weaknesses (8, nuevo), Misinformation (9, nuevo), Unbounded Consumption (10, nuevo). Las cinco nuevas entradas reflejan la madurez de los sistemas agénticos y RAG en producción. La defensa en profundidad (múltiples capas) es necesaria porque las defensas individuales se eluden con ataques adaptativos. El AI Act requiere evidencia de testing adversarial para sistemas de alto riesgo.